```
# ============================================================================
# SmartTAR STAR 1.0 Beta 1 Fix 2 XZ9 XZStable - Readable Clean
# Powered by Windows tar.exe / bsdtar only
#
# Cíl této verze:
#   - neměnit funkční logiku oproti XZStable verzi,
#   - jen lépe rozdělit kód do sekcí,
#   - zachovat XZ9, SmartXZ a stabilizaci timestampů pouze pro XZ/XZ9 bloky.
# ============================================================================

Add-Type -AssemblyName System.Windows.Forms
Add-Type -AssemblyName System.Drawing

# ============================================================================
# 01. Console handling
# ============================================================================
if (-not ("SmartTarConsoleWindow" -as [type])) {
Add-Type @"
using System;
using System.Runtime.InteropServices;
public static class SmartTarConsoleWindow {
    [DllImport("kernel32.dll")] public static extern IntPtr GetConsoleWindow();
    [DllImport("user32.dll")] public static extern bool ShowWindow(IntPtr hWnd,
int nCmdShow);
}
"@
}

$consolePtr = [SmartTarConsoleWindow]::GetConsoleWindow()
if ($consolePtr -ne [IntPtr]::Zero) {
    [SmartTarConsoleWindow]::ShowWindow($consolePtr, 0) | Out-Null
}
[System.Windows.Forms.Application]::EnableVisualStyles()

# ============================================================================
# 02. Generic helpers
# ============================================================================
function New-Point { param([int]$X,[int]$Y) return
[System.Drawing.Point]::new($X,$Y) }
function New-Size  { param([int]$W,[int]$H) return
[System.Drawing.Size]::new($W,$H) }
function Is-Blank  { param([string]$Text) return
[string]::IsNullOrWhiteSpace($Text) }

function New-UiObject {
    param([string]$Type,[hashtable]$Props)
    $obj = New-Object $Type
    foreach ($key in $Props.Keys) { $obj.$key = $Props[$key] }
    return $obj
}

function Format-Bytes {
    param([int64]$Bytes)
```

```
    if ($Bytes -ge 1GB) { return ("{0:N2} GB" -f ($Bytes / 1GB)) }
    if ($Bytes -ge 1MB) { return ("{0:N2} MB" -f ($Bytes / 1MB)) }
    if ($Bytes -ge 1KB) { return ("{0:N2} KB" -f ($Bytes / 1KB)) }
    return ("{0} B" -f $Bytes)
}

function Get-ErrorDetails {
    param($ErrorRecord)

    $lines = New-Object System.Collections.Generic.List[string]
    $lines.Add("Message:") | Out-Null
    $lines.Add([string]$ErrorRecord.Exception.Message) | Out-Null
    $lines.Add("") | Out-Null
    $lines.Add("Exception type:") | Out-Null
    $lines.Add([string]$ErrorRecord.Exception.GetType().FullName) | Out-Null

    if ($ErrorRecord.InvocationInfo) {
        $lines.Add("") | Out-Null
        $lines.Add("Position:") | Out-Null
        $lines.Add([string]$ErrorRecord.InvocationInfo.PositionMessage) |
Out-Null
    }

    if ($ErrorRecord.ScriptStackTrace) {
        $lines.Add("") | Out-Null
        $lines.Add("Script stack trace:") | Out-Null
        $lines.Add([string]$ErrorRecord.ScriptStackTrace) | Out-Null
    }

    return ($lines -join "`r`n")
}

function Get-FileSHA256 {
    param([string]$Path)
    if (-not (Test-Path -LiteralPath $Path)) { throw "Cannot hash missing file:
$Path" }
    return (Get-FileHash -LiteralPath $Path -Algorithm
SHA256).Hash.ToLowerInvariant()
}

function Get-NormalizedFullPath {
    param([string]$Path)
    if (Is-Blank $Path) { return "" }
    return [System.IO.Path]::GetFullPath($Path).TrimEnd('\','/')
}

function Get-FixedUtcTime { return
[datetime]::SpecifyKind([datetime]"2000-01-01T00:00:00",[System.DateTimeKind]::U
tc) }
function Get-FixedUtcText { return "2000-01-01T00:00:00Z" }

function Set-TreeTimestamp {
    param([string]$Path,[datetime]$Timestamp)
```

```
    if (-not (Test-Path -LiteralPath $Path)) { return }

    Get-ChildItem -LiteralPath $Path -Recurse -Force -ErrorAction
SilentlyContinue | ForEach-Object {
        try { $_.CreationTimeUtc = $Timestamp } catch {}
        try { $_.LastAccessTimeUtc = $Timestamp } catch {}
        try { $_.LastWriteTimeUtc = $Timestamp } catch {}
    }

    try {
        $root = Get-Item -LiteralPath $Path -Force
        $root.CreationTimeUtc = $Timestamp
        $root.LastAccessTimeUtc = $Timestamp
        $root.LastWriteTimeUtc = $Timestamp
    } catch {}
}

function Get-ReportPath {
    param([string]$BasePath,[string]$Kind)

    $dir = [System.IO.Path]::GetDirectoryName($BasePath)
    if (Is-Blank $dir) { $dir = (Get-Location).Path }

    $name = [System.IO.Path]::GetFileName($BasePath)
    $stamp = Get-Date -Format "yyyyMMdd_HHmmss"
    return (Join-Path $dir ("$name.$Kind.$stamp.txt"))
}

function Write-ReportFile {
    param([string]$Path,[string]$Text)
    try {
        $Text | Set-Content -LiteralPath $Path -Encoding UTF8
        return $true
    } catch {
        return $false
    }
}

# ================================================================================
# 03. Application state and UI constants
# ================================================================================
$cBg   = [System.Drawing.Color]::White
$cTxt  = [System.Drawing.ColorTranslator]::FromHtml("#2F4F4F")
$cGray = [System.Drawing.Color]::LightGray

$fNormal = [System.Drawing.Font]::new("Segoe UI",9)
$fBold   = [System.Drawing.Font]::new("Segoe
UI",9,[System.Drawing.FontStyle]::Bold)
$fItalic = [System.Drawing.Font]::new("Segoe
UI",9,[System.Drawing.FontStyle]::Italic)

$scriptDir = if ($PSScriptRoot) {
```

```
    $PSScriptRoot
} elseif ($MyInvocation.MyCommand.Path) {
    Split-Path -Parent $MyInvocation.MyCommand.Path
} else {
    (Get-Location).Path
}
if (Is-Blank $scriptDir) { $scriptDir = (Get-Location).Path }

$tarPath = Join-Path $env:SystemRoot "System32\tar.exe"
if (-not (Test-Path -LiteralPath $tarPath)) {
    $cmdTar = Get-Command "tar.exe" -ErrorAction SilentlyContinue
    if ($cmdTar -and $cmdTar.Source) { $tarPath = $cmdTar.Source }
}

$script:selectedPath = ""
$script:selectedType = ""

# ================================================================================
# 04. UI helper functions
# ================================================================================
function New-EcoLabel {
    param(
        [string]$Text,
        [int]$X,
        [int]$Y,
        [int]$W = 470,
        [int]$H = 20,
        [System.Drawing.Font]$Font = $fNormal,
        [System.Drawing.Color]$ForeColor = $cTxt
    )

    return New-UiObject "System.Windows.Forms.Label" @{
        Text      = $Text
        Location  = (New-Point $X $Y)
        Size      = (New-Size $W $H)
        Font      = $Font
        ForeColor = $ForeColor
        BackColor = $cBg
    }
}

function New-EcoButton {
    param(
        [string]$Text,
        [int]$X,
        [int]$Y,
        [int]$W,
        [int]$H,
        [System.Drawing.Font]$Font = $fNormal,
        [System.Drawing.Color]$BackColor = $cBg,
        [System.Drawing.Color]$ForeColor = [System.Drawing.Color]::Black
    )
```

```
    $button = New-UiObject "System.Windows.Forms.Button" @{
        Text                    = $Text
        Location                = (New-Point $X $Y)
        Size                    = (New-Size $W $H)
        Font                    = $Font
        BackColor               = $BackColor
        ForeColor               = $ForeColor
        UseVisualStyleBackColor = $false
    }

    try {
        $button.FlatStyle = [System.Windows.Forms.FlatStyle]::Flat
        $button.FlatAppearance.BorderSize = 1
        $button.FlatAppearance.BorderColor = $cGray
    } catch {}

    return $button
}

function New-EcoCheck {
    param([string]$Text,[int]$X,[int]$Y,[int]$W,[bool]$Checked=$true)
    return New-UiObject "System.Windows.Forms.CheckBox" @{
        Text      = $Text
        Location  = (New-Point $X $Y)
        Size      = (New-Size $W 22)
        Font      = $fNormal
        BackColor = $cBg
        ForeColor = $cTxt
        Checked   = $Checked
    }
}

function Msg {
    param(
        [string]$Message,
        [string]$Title = "SmartTAR STAR 1.0 Beta 1 Fix 2 XZ9 XZStable - Clean",
        [System.Windows.Forms.MessageBoxIcon]$Icon =
[System.Windows.Forms.MessageBoxIcon]::Information,
        [System.Windows.Forms.MessageBoxButtons]$Buttons =
[System.Windows.Forms.MessageBoxButtons]::OK
    )

    return
[System.Windows.Forms.MessageBox]::Show($Message,$Title,$Buttons,$Icon)
}

function Set-AppStatus {

param([string]$Text,[System.Drawing.Color]$Color=[System.Drawing.Color]::DimGray
)
    $lblStatus.Text = $Text
    $lblStatus.ForeColor = $Color
    $form.Refresh()
```

```
    [System.Windows.Forms.Application]::DoEvents()
}

function Start-UiWork {
    $progressBar.Visible = $true
    $progressBar.MarqueeAnimationSpeed = 25
    $form.Refresh()
    [System.Windows.Forms.Application]::DoEvents()
}

function Stop-UiWork {
    $progressBar.MarqueeAnimationSpeed = 0
    $progressBar.Visible = $false
    $form.Refresh()
}

# ================================================================================
# 05. TAR engine and method selection
# ================================================================================
function Get-TarMethods {
    return @(
        @{ Name="store";  Display="STORE";  Extension=".tar";
CreateArgs=@("-cf"); Level=$null; Algorithm="store" },
        @{ Name="gzip";   Display="GZIP";   Extension=".tar.gz";
CreateArgs=@("-czf"); Level=$null; Algorithm="gzip" },
        @{ Name="bzip2";  Display="BZIP2";  Extension=".tar.bz2";
CreateArgs=@("-cjf"); Level=$null; Algorithm="bzip2" },
        @{ Name="xz9";    Display="XZ9";    Extension=".tar.xz";
CreateArgs=@("--options","xz:compression-level=9","-cJf"); Level=9;
Algorithm="xz" },
        @{ Name="xz";     Display="XZ";     Extension=".tar.xz";
CreateArgs=@("-cJf"); Level=$null; Algorithm="xz" },
        @{ Name="zstd19"; Display="ZSTD19"; Extension=".tar.zst";
CreateArgs=@("--zstd","--options","zstd:compression-level=19","-cf"); Level=19;
Algorithm="zstd" }
    )
}

function Get-TarMethodByName {
    param([string]$Name)
    foreach ($method in Get-TarMethods) {
        if ([string]$method.Name -eq $Name) { return $method }
    }
    return $null
}

function Invoke-Tar {
    param([string]$TarPath,$TarArgs,[string]$FailMessage)

    $argList = @()
    foreach ($arg in @($TarArgs)) { $argList += [string]$arg }

    $output = & $TarPath @argList 2>&1
```

```
    if ($LASTEXITCODE -ne 0) {
        $text = ($output | Out-String).Trim()
        if (Is-Blank $text) { $text = "No tar.exe output captured." }
        throw "$FailMessage tar.exe exit code: $LASTEXITCODE`r`n$text"
    }
}

function Invoke-TarList {
    param([string]$TarPath,[string]$ArchivePath)
    $null = & $TarPath @("-tf",$ArchivePath) 2>&1
    return ($LASTEXITCODE -eq 0)
}

function Test-TarCapabilities {
    param([string]$TarPath)

    $guid = [guid]::NewGuid().ToString("N")
    $root = Join-Path $env:TEMP "smarttar_cap_$guid"
    $sample = Join-Path $root "sample"
    $extract = Join-Path $root "extract"

    New-Item -ItemType Directory -Path $sample,$extract -Force | Out-Null
    "SmartTAR capability test" | Set-Content -LiteralPath (Join-Path $sample
"sample.txt") -Encoding UTF8

    $result = @{}

    foreach ($method in Get-TarMethods) {
        $name = [string]$method.Name
        $archivePath = Join-Path $root ("test" + [string]$method.Extension)
        $extractDir = Join-Path $extract $name
        New-Item -ItemType Directory -Path $extractDir -Force | Out-Null

        $ok = $false
        try {
            $args = @()
            $args += $method.CreateArgs
            $args += $archivePath
            $args += "-C"
            $args += $sample
            $args += "sample.txt"

            $null = & $TarPath @args 2>&1
            if ($LASTEXITCODE -eq 0 -and (Test-Path -LiteralPath $archivePath))
{
                $null = & $TarPath @("-xf",$archivePath,"-C",$extractDir) 2>&1
                if ($LASTEXITCODE -eq 0 -and (Test-Path -LiteralPath (Join-Path
$extractDir "sample.txt"))) { $ok = $true }
            }
        } catch {
            $ok = $false
        }
```

```
        $result[$name] = $ok
    }

    Remove-Item -LiteralPath $root -Recurse -Force -ErrorAction SilentlyContinue
    return $result
}

function Select-BestCompressedMethod {
    param([hashtable]$Capabilities)
    foreach ($name in @("xz9","xz","bzip2","gzip","store")) {
        if ($Capabilities.ContainsKey($name) -and $Capabilities[$name]) { return
Get-TarMethodByName $name }
    }
    throw "No usable tar method was found."
}

function Select-XzOrBest {
    param([hashtable]$Capabilities)
    if ($Capabilities.ContainsKey("xz9") -and $Capabilities["xz9"]) { return
Get-TarMethodByName "xz9" }
    if ($Capabilities.ContainsKey("xz")  -and $Capabilities["xz"])  { return
Get-TarMethodByName "xz" }
    return Select-BestCompressedMethod $Capabilities
}

function Select-ZstdOrBest {
    param([hashtable]$Capabilities)
    if ($Capabilities.ContainsKey("zstd19") -and $Capabilities["zstd19"]) {
return Get-TarMethodByName "zstd19" }
    return Select-BestCompressedMethod $Capabilities
}

function Select-StoreMethod {
    param([hashtable]$Capabilities)
    if ($Capabilities.ContainsKey("store") -and $Capabilities["store"]) { return
Get-TarMethodByName "store" }
    return Select-BestCompressedMethod $Capabilities
}

# ============================================================================
# 06. Smart grouping and source profile
# ============================================================================
function Get-SmartGroupName {
    param([string]$FilePath)

    $ext = [System.IO.Path]::GetExtension($FilePath).ToLowerInvariant()

    $textExt       =
@(".txt",".csv",".json",".xml",".log",".ini",".cfg",".md",".sql",".ps1",".bat","
.cmd",".html",".htm",".css",".js",".ts",".yml",".yaml",".toml",".reg",".inf",".s
rt",".vtt")
    $binaryExt     =
@(".bin",".dat",".db",".sqlite",".sqlite3",".pak",".asset",".res",".idx",".map",
```

```
".cache",".blob")
    $executableExt =
@(".exe",".dll",".sys",".ocx",".msi",".msp",".scr",".com",".drv",".efi")
    $diskImageExt  = @(".iso",".img",".vhd",".vhdx")
    $mediaExt      =
@(".jpg",".jpeg",".png",".gif",".webp",".bmp",".tif",".tiff",".ico",".mp3",".wav
",".flac",".aac",".ogg",".wma",".mp4",".mkv",".avi",".mov",".wmv",".webm",".pdf"
,".heic",".avif")
    $archiveExt    =
@(".zip",".7z",".rar",".gz",".bz2",".xz",".zst",".tar",".tgz",".tbz2",".txz",".c
ab",".jar",".war",".ear",".sarc",".star",".docx",".xlsx",".pptx",".odt",".ods","
.odp",".apk",".epub",".vsix",".nupkg")

    if ($textExt -contains $ext)       { return "text" }
    if ($diskImageExt -contains $ext)  { return "diskimage" }
    if ($binaryExt -contains $ext)     { return "binary" }
    if ($executableExt -contains $ext) { return "executable" }
    if ($mediaExt -contains $ext)      { return "media" }
    if ($archiveExt -contains $ext)    { return "archives" }
    return "unknown"
}

function Get-ModeGroupName {
    param([string]$Mode,[string]$SmartGroup)

    if ($Mode -eq "Solid")   { return "solid" }
    if ($Mode -eq "SmartXZ") { return $SmartGroup }

    if ($Mode -eq "Hybrid") {
        if ($SmartGroup -eq "diskimage") { return "diskimage" }
        if ($SmartGroup -eq "media" -or $SmartGroup -eq "archives") { return
"stored" }
        return "compressible"
    }

    return $SmartGroup
}

function Get-RelativePathFromBase {
    param([string]$BasePath,[string]$FullPath)

    $baseFull = [System.IO.Path]::GetFullPath($BasePath)
    $pathFull = [System.IO.Path]::GetFullPath($FullPath)
    $baseClean = $baseFull -replace '[\/]+$',''
    $prefix = $baseClean + [System.IO.Path]::DirectorySeparatorChar

    if ($pathFull.ToLowerInvariant().StartsWith($prefix.ToLowerInvariant())) {
return $pathFull.Substring($prefix.Length) }
    return (Split-Path -Leaf $pathFull)
}

function Get-SourceSize {
    param([string]$Path)
```

```
    try {
        $item = Get-Item -LiteralPath $Path -ErrorAction Stop
        if (-not $item.PSIsContainer) { return [int64]$item.Length }
        $sum = [int64]0
        Get-ChildItem -LiteralPath $Path -Recurse -Force -ErrorAction
SilentlyContinue | ForEach-Object {
            if (-not $_.PSIsContainer) { $sum += [int64]$_.Length }
        }
        return $sum
    } catch {
        return [int64]0
    }
}

function Get-SortedSourceFiles {
    param($SourceItem,[string]$Source,[string]$BaseRoot="")

    if (-not $SourceItem.PSIsContainer) { return @($SourceItem) }

    if (Is-Blank $BaseRoot) {
        return @(Get-ChildItem -LiteralPath $Source -File -Recurse -Force
-ErrorAction SilentlyContinue | Sort-Object
@{Expression={$_.FullName.ToLowerInvariant()}},@{Expression={$_.FullName}})
    }

    return @(Get-ChildItem -LiteralPath $Source -File -Recurse -Force
-ErrorAction SilentlyContinue | Sort-Object
@{Expression={(Get-RelativePathFromBase $BaseRoot
$_.FullName).ToLowerInvariant()}},@{Expression={Get-RelativePathFromBase
$BaseRoot $_.FullName}})
}

function Get-SourceProfile {
    param($SourceItem,[string]$Source)

    $profile =
@{text=[int64]0;binary=[int64]0;executable=[int64]0;diskimage=[int64]0;media=[in
t64]0;archives=[int64]0;unknown=[int64]0;files=0}

    foreach ($file in (Get-SortedSourceFiles $SourceItem $Source)) {
        $group = Get-SmartGroupName $file.FullName
        $profile[$group] = [int64]$profile[$group] + [int64]$file.Length
        $profile.files++
    }

    return $profile
}

function Select-AutoSolidMethod {
    param([hashtable]$Capabilities,[hashtable]$Profile)

    $zstd = Select-ZstdOrBest $Capabilities
```

```
    $xz   = Select-XzOrBest $Capabilities

    if ($Capabilities.ContainsKey("zstd19") -and $Capabilities["zstd19"]) {
        $binaryLike = [int64]$Profile.binary + [int64]$Profile.executable +
[int64]$Profile.diskimage
        if ($binaryLike -gt [int64]$Profile.text) { return $zstd }
    }

    return $xz
}

function New-GroupInfo {
    param([string]$Name,[hashtable]$Method,[string]$Stage,[string]$Reason)
    return
@{Name=$Name;Method=$Method;Reason=$Reason;FileCount=0;DirCount=0;Bytes=[int64]0
;Stage=$Stage}
}

function New-ArchiveGroups {

param([string]$Mode,[hashtable]$Capabilities,[hashtable]$Profile,[string]$Stagin
gRoot)

    $store = Select-StoreMethod $Capabilities
    $xz    = Select-XzOrBest $Capabilities
    $zstd  = Select-ZstdOrBest $Capabilities
    $groups = [ordered]@{}

    switch ($Mode) {
        "Solid" {
            $solid = Select-AutoSolidMethod $Capabilities $Profile
            $groups["solid"] = New-GroupInfo "solid" $solid (Join-Path
$StagingRoot "solid") "Auto solid method selected from source profile."
        }
        "SmartXZ" {
            $groups["structure"]  = New-GroupInfo "structure"  $store (Join-Path
$StagingRoot "structure")  "Directory structure, no compression needed."
            $groups["text"]       = New-GroupInfo "text"       $xz    (Join-Path
$StagingRoot "text")       "Smart XZ mode: text-like data grouped and compressed
with XZ9 when available."
            $groups["binary"]     = New-GroupInfo "binary"     $xz    (Join-Path
$StagingRoot "binary")     "Smart XZ mode: binary data grouped and compressed
with XZ9 when available."
            $groups["executable"] = New-GroupInfo "executable" $xz    (Join-Path
$StagingRoot "executable") "Smart XZ mode: executable data grouped and
compressed with XZ9 when available."
            $groups["diskimage"]  = New-GroupInfo "diskimage"  $xz    (Join-Path
$StagingRoot "diskimage")  "Smart XZ mode: disk image data grouped and
compressed with XZ9 when available."
            $groups["media"]      = New-GroupInfo "media"      $store (Join-Path
$StagingRoot "media")      "Media is usually already compressed, stored."
            $groups["archives"]   = New-GroupInfo "archives"   $store (Join-Path
$StagingRoot "archives")   "Archive-like data is usually already compressed,
```

```
stored."
            $groups["unknown"]    = New-GroupInfo "unknown"    $xz    (Join-Path
$StagingRoot "unknown")    "Smart XZ mode: unknown data grouped and compressed
with XZ9 when available."
        }
        "Smart" {
            $groups["structure"]  = New-GroupInfo "structure"  $store (Join-Path
$StagingRoot "structure")  "Directory structure, no compression needed."
            $groups["text"]       = New-GroupInfo "text"       $xz    (Join-Path
$StagingRoot "text")       "Text-like data prefers XZ9 when available."
            $groups["binary"]     = New-GroupInfo "binary"     $zstd  (Join-Path
$StagingRoot "binary")     "Binary data prefers ZSTD19 when available."
            $groups["executable"] = New-GroupInfo "executable" $zstd  (Join-Path
$StagingRoot "executable") "Executable data prefers ZSTD19 when available."
            $groups["diskimage"]  = New-GroupInfo "diskimage"  $zstd  (Join-Path
$StagingRoot "diskimage")  "Disk image data prefers ZSTD19 when available."
            $groups["media"]      = New-GroupInfo "media"      $store (Join-Path
$StagingRoot "media")      "Media is usually already compressed, stored."
            $groups["archives"]   = New-GroupInfo "archives"   $store (Join-Path
$StagingRoot "archives")   "Archive-like data is usually already compressed,
stored."
            $groups["unknown"]    = New-GroupInfo "unknown"    $xz    (Join-Path
$StagingRoot "unknown")    "Unknown data fallback prefers XZ9 when available."
        }
        default {
            $groups["structure"]    = New-GroupInfo "structure"    $store
(Join-Path $StagingRoot "structure")    "Directory structure, no compression
needed."
            $groups["compressible"] = New-GroupInfo "compressible" $xz
(Join-Path $StagingRoot "compressible") "General compressible block prefers XZ9
when available."
            $groups["diskimage"]    = New-GroupInfo "diskimage"    $zstd
(Join-Path $StagingRoot "diskimage")    "Disk image data prefers ZSTD19 when
available."
            $groups["stored"]       = New-GroupInfo "stored"       $store
(Join-Path $StagingRoot "stored")       "Media and archive-like data is stored."
        }
    }

    return $groups
}

# ============================================================================
# 07. XZ timestamp stabilization
# ============================================================================
function Test-GroupUsesXz {
    param($Group)
    try { return ([string]$Group.Method.Algorithm -eq "xz") } catch { return
$false }
}

function Test-AnyXzGroup {
    param([hashtable]$Groups)
```

```
    foreach ($key in $Groups.Keys) {
        if (Test-GroupUsesXz $Groups[$key]) { return $true }
    }
    return $false
}

function Set-XzGroupTimestamps {
    param([hashtable]$Groups)
    $fixedTime = Get-FixedUtcTime
    foreach ($key in $Groups.Keys) {
        if (Test-GroupUsesXz $Groups[$key]) { Set-TreeTimestamp
$Groups[$key].Stage $fixedTime }
    }
}

# ============================================================================
# 08. Staging, blocks and manifest
# ============================================================================
function Copy-FileToGroupStage {
    param([string]$SourceFile,[string]$RelativePath,[string]$GroupStageRoot)
    $target = Join-Path $GroupStageRoot $RelativePath
    $dir = Split-Path -Parent $target
    if (-not (Test-Path -LiteralPath $dir)) { New-Item -ItemType Directory -Path
$dir -Force | Out-Null }
    Copy-Item -LiteralPath $SourceFile -Destination $target -Force
}

function Create-DirInGroupStage {
    param([string]$RelativePath,[string]$GroupStageRoot)
    New-Item -ItemType Directory -Path (Join-Path $GroupStageRoot $RelativePath)
-Force | Out-Null
}

function Create-BlockFromStage {

param([string]$TarPath,[string]$StagePath,[string]$BlockPath,[hashtable]$Method)
    if (-not (Test-Path -LiteralPath $StagePath)) { throw "Stage path does not
exist: $StagePath" }
    $args = @()
    $args += $Method.CreateArgs
    $args += $BlockPath
    $args += "-C"
    $args += $StagePath
    $args += "."
    Invoke-Tar -TarPath $TarPath -TarArgs $args -FailMessage "Block creation
failed: $BlockPath."
}

function Stage-Directories {

param($SourceItem,[string]$Source,[string]$BaseRoot,[string]$Mode,[hashtable]$Gr
oups)
    if (-not $SourceItem.PSIsContainer) { return }
```

```
    $targetGroup = if ($Mode -eq "Solid") { "solid" } else { "structure" }
    $rootRel = Get-RelativePathFromBase $BaseRoot $Source
    Create-DirInGroupStage $rootRel $Groups[$targetGroup].Stage
    $Groups[$targetGroup].DirCount = [int]$Groups[$targetGroup].DirCount + 1

    Get-ChildItem -LiteralPath $Source -Directory -Recurse -Force -ErrorAction
SilentlyContinue |
        Sort-Object @{Expression={(Get-RelativePathFromBase $BaseRoot
$_.FullName).ToLowerInvariant()}},@{Expression={Get-RelativePathFromBase
$BaseRoot $_.FullName}} |
        ForEach-Object {
            $rel = Get-RelativePathFromBase $BaseRoot $_.FullName
            Create-DirInGroupStage $rel $Groups[$targetGroup].Stage
            $Groups[$targetGroup].DirCount = [int]$Groups[$targetGroup].DirCount
+ 1
        }
}

function Stage-Files {

param($SourceItem,[string]$Source,[string]$BaseRoot,[string]$Mode,[hashtable]$Gr
oups)
    Set-AppStatus "Planning and sorting files into $Mode mode..."
([System.Drawing.Color]::DarkOrange)
    foreach ($file in (Get-SortedSourceFiles $SourceItem $Source $BaseRoot)) {
        $relative = Get-RelativePathFromBase $BaseRoot $file.FullName
        $smartGroup = Get-SmartGroupName $file.FullName
        $groupName = Get-ModeGroupName $Mode $smartGroup
        Copy-FileToGroupStage $file.FullName $relative $Groups[$groupName].Stage
        $Groups[$groupName].FileCount = [int]$Groups[$groupName].FileCount + 1
        $Groups[$groupName].Bytes = [int64]$Groups[$groupName].Bytes +
[int64]$file.Length
    }
}

function Build-Blocks {
    param([string]$TarPath,[hashtable]$Groups,[string]$BlocksDir)
    Set-AppStatus "Creating internal blocks..."
([System.Drawing.Color]::DarkOrange)
    $list = @()
    $index = 1
    foreach ($groupName in $Groups.Keys) {
        $group = $Groups[$groupName]
        if (-not (([int]$group.FileCount -gt 0) -or ([int]$group.DirCount -gt
0))) { continue }
        $blockId = "{0:D6}" -f $index
        $method = $group.Method
        $cleanGroup = [string]$group.Name
        $extension = [string]$method.Extension
        $blockName = "$blockId`_$cleanGroup$extension"
        $blockPath = Join-Path $BlocksDir $blockName
        Create-BlockFromStage $TarPath $group.Stage $blockPath $method
        $blockItem = Get-Item -LiteralPath $blockPath
```

```
        $list += [ordered]@{
            id          = $blockId
            group       = $cleanGroup
            path        = "blocks/$blockName"
            container   = "tar"
            compression = [string]$method.Algorithm
            method      = [string]$method.Name
            display     = [string]$method.Display
            level       = $method.Level
            extension   = $extension
            tarArgs     = ($method.CreateArgs -join " ")
            reason      = [string]$group.Reason
            fileCount   = [int]$group.FileCount
            dirCount    = [int]$group.DirCount
            sourceBytes = [int64]$group.Bytes
            sizeBytes   = [int64]$blockItem.Length
            sha256      = Get-FileSHA256 $blockPath
        }
        $index++
    }
    return $list
}

function Write-Manifest {
    param([string]$Path,$Data)
    $Data | ConvertTo-Json -Depth 30 | Set-Content -LiteralPath $Path -Encoding
UTF8
}

function Build-Manifest {

param([string]$Source,$SourceItem,[string]$SourceLeaf,[string]$Mode,[hashtable]$
CapabilityMap,[hashtable]$Profile,$BlockList)
    $name = $SourceLeaf
    if ($SourceLeaf -eq ".") { $name = Split-Path -Leaf ($Source -replace
'[\/]+$','') }
    $type = if ($SourceItem.PSIsContainer) { "Folder" } else { "File" }
    $xzBlocks = @(@($BlockList) | Where-Object { [string]$_.compression -eq "xz"
})
    $detEnabled = ($xzBlocks.Count -gt 0)
    $detTime = if ($detEnabled) { Get-FixedUtcText } else { "not-normalized" }
    $detScope = if ($detEnabled) { "XZ/XZ9-blocks" } else { "none" }

    return [ordered]@{
        format          = "STAR"
        formatVersion   = 1
        tool            = "SmartTAR"
        version         = 18
        toolVersion     = "1.0-beta1-fix2-xz9-xzstable-clean"
        createdUtc      =
(Get-Date).ToUniversalTime().ToString("yyyy-MM-ddTHH:mm:ssZ")
        engine          = "Windows tar.exe"
        model           = "auto-planner-v1.0-beta1-fix2-xz9-xzstable-clean"
```

```
        compressionMode = $Mode
        sourceName      = $name
        sourceType      = $type
        sourceBytes     = Get-SourceSize $Source
        grouping        = "auto-planned"
        planning        = [ordered]@{
            strategy          = "auto"
            rulesVersion      = "smart-plan-v1"
            xzMaxLevel        = 9
            zstdMaxLevel      = 19
            defaultMode       = "Hybrid"
            experimentalModes = @("SmartXZ")
            defaultExtension  = ".star"
            legacyExtensions  = @(".sarc.tar")
            design            = "Transparent TAR outer container with SmartTAR
block manifest."
        }
        deterministicMetadata = [ordered]@{
            enabled      = $detEnabled
            timestampUtc = $detTime
            scope        = $detScope
            note         = "Timestamps are normalized only for block stages
using XZ/XZ9. STORE and ZSTD stages are not normalized."
        }
        sourceProfile = $Profile
        capabilities  = [ordered]@{
            store  = [bool]$CapabilityMap.store
            gzip   = [bool]$CapabilityMap.gzip
            bzip2  = [bool]$CapabilityMap.bzip2
            xz9    = [bool]$CapabilityMap.xz9
            xz     = [bool]$CapabilityMap.xz
            zstd19 = [bool]$CapabilityMap.zstd19
        }
        blocks = @($BlockList)
        manualRecovery = @(
            "tar -xf archive.star -C outer",
            "tar -xf outer\blocks\000001_solid.tar.xz -C restore",
            "If your tool does not recognize .star, rename archive.star to
archive.tar and inspect it manually."
        )
    }
}

# ============================================================================
# 09. Archive safety, extraction and verify
# ============================================================================
function Read-OuterManifest {
    param([string]$OuterRoot)
    $manifestPath = Join-Path $OuterRoot "manifest.json"
    if (-not (Test-Path -LiteralPath $manifestPath)) { throw "manifest.json was
not found. This does not look like a SmartTAR archive." }
    $manifest = Get-Content -LiteralPath $manifestPath -Raw -Encoding UTF8 |
ConvertFrom-Json
```

```
    if ($manifest.format -ne "STAR" -and $manifest.format -ne "SARC" -and
$manifest.format -ne "SmartTarArc") { throw "Invalid archive format. Expected
STAR, SARC or SmartTarArc." }
    return $manifest
}

function Test-RelativePathSafe {
    param([string]$PathText)
    if (Is-Blank $PathText) { return $false }
    $path = ([string]$PathText).Replace('\','/')
    if ($path -eq "." -or $path -eq "./") { return $true }
    if ($path -match '^[a-zA-Z]:') { return $false }
    if ($path.StartsWith("/") -or $path.StartsWith("//")) { return $false }
    $parts = @($path.Split('/') | Where-Object { -not
[string]::IsNullOrWhiteSpace($_) -and $_ -ne "." })
    foreach ($part in $parts) { if ($part -eq "..") { return $false } }
    return $true
}

function Resolve-SafeBlockPath {
    param([string]$OuterRoot,[string]$RelativeBlockPath)
    if (-not (Test-RelativePathSafe $RelativeBlockPath)) { throw "Unsafe block
path detected in manifest: $RelativeBlockPath" }
    $separator = [string][System.IO.Path]::DirectorySeparatorChar
    return (Join-Path $OuterRoot ($RelativeBlockPath -replace '/', $separator))
}

function Test-ArchiveEntriesSafe {
    param([string]$TarPath,[string]$ArchivePath)
    $entries = & $TarPath @("-tf",$ArchivePath) 2>&1
    if ($LASTEXITCODE -ne 0) {
        $text = ($entries | Out-String).Trim()
        throw "Cannot list TAR block before extraction: $ArchivePath`r`n$text"
    }
    foreach ($entry in $entries) {
        if (-not (Test-RelativePathSafe ([string]$entry))) { throw "Unsafe path
inside TAR block detected: $entry" }
    }
}

function Extract-Blocks {

param([string]$TarPath,[string]$OuterRoot,$Blocks,[string]$DestinationFolder,[bo
ol]$SkipFailedBlocks=$false)
    Set-AppStatus "Extracting internal blocks..."
([System.Drawing.Color]::DarkOrange)
    foreach ($block in $Blocks) {
        $relativePath = [string]$block.path
        $blockPath = Resolve-SafeBlockPath $OuterRoot $relativePath
        if (-not (Test-Path -LiteralPath $blockPath)) {
            if ($SkipFailedBlocks) { continue }
            throw "Block file was not found: $relativePath"
        }
```

```
        if ($block.sha256) {
            $actualHash = Get-FileSHA256 $blockPath
            if ($actualHash -ne ([string]$block.sha256).ToLowerInvariant()) {
                if ($SkipFailedBlocks) { continue }
                throw "Block SHA256 mismatch: $relativePath"
            }
        }
        Test-ArchiveEntriesSafe $TarPath $blockPath
        Invoke-Tar $TarPath @("-xf",$blockPath,"-C",$DestinationFolder) "Block
extraction failed: $relativePath."
    }
}

function Get-UsedMethodReport {
    param($Blocks,$Manifest)
    $methodMap = @{}
    foreach ($block in @($Blocks)) {
        $display = [string]$block.display
        $compression = [string]$block.compression
        if (Is-Blank $display) { $display = [string]$block.method }
        if (Is-Blank $compression) { $compression = "n/a" }
        $key = "{0}/{1}" -f $display,$compression
        if (-not $methodMap.ContainsKey($key)) { $methodMap[$key] = 0 }
        $methodMap[$key] = [int]$methodMap[$key] + 1
    }
    $used = if ($methodMap.Count -gt 0) { (($methodMap.Keys | Sort-Object |
ForEach-Object { "{0} x{1}" -f $_,$methodMap[$_] }) -join ", ") } else { "n/a" }
    $zstdBlocks = @(@($Blocks) | Where-Object { ([string]$_.compression -eq
"zstd") -or ([string]$_.method -like "zstd*") })
    $configured = "n/a"
    try { if ($Manifest.planning -and
($Manifest.planning.PSObject.Properties.Name -contains "zstdMaxLevel")) {
$configured = [string]$Manifest.planning.zstdMaxLevel } } catch {}
    $zstd = if ($zstdBlocks.Count -gt 0) { "used in $($zstdBlocks.Count)
block(s), configured max level: $configured" } else { "not used" }
    return @{Used=$used;Zstd=$zstd}
}

function Get-DeterministicReport {
    param($Manifest)
    $enabled="n/a"; $stamp="n/a"; $scope="n/a"
    try {
        if ($Manifest.deterministicMetadata) {
            $enabled = if ([bool]$Manifest.deterministicMetadata.enabled) {
"yes" } else { "no" }
            $stamp = [string]$Manifest.deterministicMetadata.timestampUtc
            $scope = [string]$Manifest.deterministicMetadata.scope
        }
    } catch {}
    return @{Enabled=$enabled;Timestamp=$stamp;Scope=$scope}
}

function Verify-SmartArchive {
```

```
    param([string]$TarPath,[string]$ArchivePath)
    if (-not (Test-Path -LiteralPath $TarPath)) { throw "tar.exe was not found."
}
    if (-not (Test-Path -LiteralPath $ArchivePath)) { throw "Archive path does
not exist." }

    $guid = [guid]::NewGuid().ToString("N")
    $work = Join-Path $env:TEMP "smarttar_verify_$guid"
    $outer = Join-Path $work "outer"
    New-Item -ItemType Directory -Path $outer -Force | Out-Null

    try {
        Set-AppStatus "Verifying outer container..."
([System.Drawing.Color]::DarkOrange)
        Invoke-Tar $TarPath @("-xf",$ArchivePath,"-C",$outer) "Outer .star
verification failed."
        $manifest = Read-OuterManifest $outer
        $blocks = @($manifest.blocks)
        if ($blocks.Count -lt 1) { throw "Manifest does not contain any blocks."
}

        $methodReport = Get-UsedMethodReport $blocks $manifest
        $detReport = Get-DeterministicReport $manifest
        $ok=0; $fail=0; $total=[int64]0; $lines=@()

        Set-AppStatus "Verifying internal blocks..."
([System.Drawing.Color]::DarkOrange)
        foreach ($block in $blocks) {
            $relativePath = [string]$block.path
            $blockPath = Resolve-SafeBlockPath $outer $relativePath
            if (-not (Test-Path -LiteralPath $blockPath)) {
                $fail++
                $lines += ("  {0} | {1} | {2} | MISSING | {3}" -f
$block.id,$block.group,$block.display,$relativePath)
                continue
            }
            $listed = Invoke-TarList $TarPath $blockPath
            $expectedHash = [string]$block.sha256
            $hashOk = $true
            if (-not (Is-Blank $expectedHash)) {
                $actualHash = Get-FileSHA256 $blockPath
                $hashOk = ($actualHash -eq $expectedHash.ToLowerInvariant())
            }
            $actualSize = [int64](Get-Item -LiteralPath $blockPath).Length
            $expectedSize = [int64]$block.sizeBytes
            $total += $actualSize
            if ($listed -and $hashOk) { $ok++ } else { $fail++ }
            if ($listed -and $hashOk) { $status="OK" } elseif ($listed -and -not
$hashOk) { $status="HASH FAIL" } else { $status="FAIL" }
            $sizeInfo = Format-Bytes $actualSize
            if ($expectedSize -gt 0 -and $expectedSize -ne $actualSize) {
$sizeInfo = "$sizeInfo / expected $(Format-Bytes $expectedSize)" }
            $hashInfo = ""
```

```
            if (-not (Is-Blank $expectedHash)) { $hashInfo = if ($hashOk) { " | SHA256 OK" } else { " | SHA256 FAIL" } }
            $reason = [string]$block.reason
            if (Is-Blank $reason) { $reason = "n/a" }
            $lines += ("  {0} | {1} | {2} | {3} | {4}{5} | {6}" -f $block.id,$block.group,$block.display,$status,$sizeInfo,$hashInfo,$reason)
        }

        $verificationStatus = if ($fail -gt 0) { "Archive verification PARTIAL / FAILED (Some blocks failed, healthy blocks may still be recoverable)" } else { "Archive verification OK (Integrity verified)" }
        $archiveBytes = [int64](Get-Item -LiteralPath $ArchivePath).Length
        return @"
$verificationStatus

Format: $($manifest.format)
Tool: $($manifest.tool)
Version: $($manifest.toolVersion)
Model: $($manifest.model)
Mode: $($manifest.compressionMode)
Planning: $($manifest.planning.strategy)
Used methods: $($methodReport.Used)
ZSTD: $($methodReport.Zstd)
Deterministic metadata: $($detReport.Enabled)
Fixed timestamp UTC: $($detReport.Timestamp)
Deterministic scope: $($detReport.Scope)
Blocks: $($blocks.Count)
Blocks OK: $ok
Blocks failed: $fail
Files declared: $(($blocks | Measure-Object -Property fileCount -Sum).Sum)
Dirs declared: $(($blocks | Measure-Object -Property dirCount -Sum).Sum)
Source size declared: $(Format-Bytes ([int64]$manifest.sourceBytes))
Archive size: $(Format-Bytes $archiveBytes)
Internal block bytes: $(Format-Bytes $total)

Blocks:
$($lines -join "`r`n")
"@
    } finally {
        if (Test-Path -LiteralPath $work) { Remove-Item -LiteralPath $work -Recurse -Force -ErrorAction SilentlyContinue }
    }
}

function Get-ArchiveSummary {
    param([string]$TarPath,[string]$ArchivePath,[string]$SourcePath)
    $sourceBytes = Get-SourceSize $SourcePath
    $archiveBytes = [int64](Get-Item -LiteralPath $ArchivePath).Length
    $ratio="n/a"; $saved="n/a"
    if ($sourceBytes -gt 0 -and $archiveBytes -gt 0) {
        $ratio = "{0:N2} %" -f (($archiveBytes / $sourceBytes) * 100)
        $saved = "{0:N2} %" -f ((1 - ($archiveBytes / $sourceBytes)) * 100)
    }
```

```
    $verify = Verify-SmartArchive $TarPath $ArchivePath
    return @"
Archive created successfully.

Source size: $(Format-Bytes $sourceBytes)
Archive size: $(Format-Bytes $archiveBytes)
Ratio: $ratio
Saved: $saved

$verify
"@
}

# ============================================================================
# 10. Core operations
# ============================================================================
function Compress-SmartArchive {
    param([string]$TarPath,[string]$Source,[string]$Destination,[string]$Mode)
    if (-not (Test-Path -LiteralPath $TarPath)) { throw "tar.exe was not found."
}
    if (-not (Test-Path -LiteralPath $Source)) { throw "Source path does not
exist." }
    if ((Get-NormalizedFullPath $Source).ToLowerInvariant() -eq
(Get-NormalizedFullPath $Destination).ToLowerInvariant()) { throw "Destination
cannot be the same as source." }
    if (Test-Path -LiteralPath $Destination) { Remove-Item -LiteralPath
$Destination -Force }
    if (Is-Blank $Mode) { $Mode = "Hybrid" }
    if ($Mode -ne "Solid" -and $Mode -ne "Hybrid" -and $Mode -ne "Smart" -and
$Mode -ne "SmartXZ") { $Mode = "Hybrid" }

    Set-AppStatus "Checking TAR capabilities..."
([System.Drawing.Color]::DarkOrange)
    $capabilities = Test-TarCapabilities $TarPath
    if (-not $capabilities.store) { throw "No usable tar store method was
found." }

    $guid = [guid]::NewGuid().ToString("N")
    $work = Join-Path $env:TEMP "smarttar_$guid"
    $blocksDir = Join-Path $work "blocks"
    $stageRoot = Join-Path $work "staging"
    New-Item -ItemType Directory -Path $work,$blocksDir,$stageRoot -Force |
Out-Null

    try {
        Set-AppStatus "Analyzing source and creating compression plan..."
([System.Drawing.Color]::DarkOrange)
        $sourceItem = Get-Item -LiteralPath $Source -Force
        $sourceParent = Split-Path -Parent $Source
        $sourceLeaf = Split-Path -Leaf $Source
        if (Is-Blank $sourceParent) { $sourceParent = (Get-Location).Path }
        if (Is-Blank $sourceLeaf) { $sourceParent = $Source; $sourceLeaf = "." }
```

```
        $profile = Get-SourceProfile $sourceItem $Source
        $groups = New-ArchiveGroups $Mode $capabilities $profile $stageRoot
        foreach ($key in $groups.Keys) { New-Item -ItemType Directory -Path
$groups[$key].Stage -Force | Out-Null }

        Stage-Directories $sourceItem $Source $sourceParent $Mode $groups
        Stage-Files $sourceItem $Source $sourceParent $Mode $groups

        $hasXz = Test-AnyXzGroup $groups
        if ($hasXz) { Set-XzGroupTimestamps $groups }

        $blockList = Build-Blocks $TarPath $groups $blocksDir
        if ($blockList.Count -lt 1) { throw "No blocks were created. Source may
be empty or inaccessible." }

        $manifest = Build-Manifest $Source $sourceItem $sourceLeaf $Mode
$capabilities $profile $blockList
        Write-Manifest (Join-Path $work "manifest.json") $manifest
        if ($hasXz) { Set-TreeTimestamp $work (Get-FixedUtcTime) }

        Set-AppStatus "Creating outer .star container..."
([System.Drawing.Color]::DarkOrange)
        Invoke-Tar $TarPath
@("-cf",$Destination,"-C",$work,"manifest.json","blocks") "Outer .star archive
creation failed."
        if (-not (Test-Path -LiteralPath $Destination)) { throw "Output archive
was not created." }
    } finally {
        if (Test-Path -LiteralPath $work) { Remove-Item -LiteralPath $work
-Recurse -Force -ErrorAction SilentlyContinue }
    }
}

function Extract-SmartArchive {
    param([string]$TarPath,[string]$ArchivePath,[string]$DestinationFolder)
    if (-not (Test-Path -LiteralPath $TarPath)) { throw "tar.exe was not found."
}
    if (-not (Test-Path -LiteralPath $ArchivePath)) { throw "Archive path does
not exist." }
    if (Is-Blank $DestinationFolder) { throw "Destination folder is empty." }
    if (-not (Test-Path -LiteralPath $DestinationFolder)) { New-Item -ItemType
Directory -Path $DestinationFolder -Force | Out-Null }

    $guid = [guid]::NewGuid().ToString("N")
    $work = Join-Path $env:TEMP "smarttar_extract_$guid"
    $outer = Join-Path $work "outer"
    New-Item -ItemType Directory -Path $outer -Force | Out-Null

    try {
        Set-AppStatus "Extracting outer container..."
([System.Drawing.Color]::DarkOrange)
        Invoke-Tar $TarPath @("-xf",$ArchivePath,"-C",$outer) "Outer .star
extraction failed."
```

```
        $manifest = Read-OuterManifest $outer
        Extract-Blocks $TarPath $outer @($manifest.blocks) $DestinationFolder
    } finally {
        if (Test-Path -LiteralPath $work) { Remove-Item -LiteralPath $work
-Recurse -Force -ErrorAction SilentlyContinue }
    }
}

# ============================================================================
# 11. Path helpers
# ============================================================================
function Is-SmartArchivePath {
    param([string]$Path)
    if (Is-Blank $Path) { return $false }
    return ([System.IO.Path]::GetFileName($Path) -match
'(?i)(\.star|\.sarc\.tar)$')
}

function Ensure-StarExtension {
    param([string]$Path)
    if (Is-Blank $Path) { return $Path }
    if ($Path -match '(?i)(\.star|\.sarc\.tar)$') { return $Path }
    return ($Path + ".star")
}

function Get-DefaultArchiveBaseName {
    param([string]$Path,[string]$Type)
    $leaf = Split-Path -Leaf $Path
    if (Is-Blank $leaf) { return "archive_{0}" -f (Get-Date -Format
"yyyyMMdd_HHmmss") }
    if ($Type -eq "Folder") { return $leaf }
    return [System.IO.Path]::GetFileNameWithoutExtension($leaf)
}

function Get-DefaultExtractBaseName {
    param([string]$Path)
    $name = [System.IO.Path]::GetFileName($Path)
    if (Is-Blank $name) { return "extracted_{0}" -f (Get-Date -Format
"yyyyMMdd_HHmmss") }
    if ($name -match '^(.*)\.star$') { return $matches[1] }
    if ($name -match '^(.*)\.sarc\.tar$') { return $matches[1] }
    return [System.IO.Path]::GetFileNameWithoutExtension($name)
}

function Get-SelectedCompressionMode {
    $text = [string]$cmbMode.SelectedItem
    if ($text -like "Smart XZ*") { return "SmartXZ" }
    if ($text -like "Smart*")    { return "Smart" }
    if ($text -like "Solid*")    { return "Solid" }
    return "Hybrid"
}

function Set-DefaultTarget {
```

```
    if (Is-Blank $script:selectedPath) { return }
    $parent = Split-Path -Parent $script:selectedPath
    if (Is-Blank $parent) { $parent = $scriptDir }
    if ($script:selectedType -eq "File" -and (Is-SmartArchivePath
$script:selectedPath)) {
        $txtTarget.Text = Join-Path $parent ((Get-DefaultExtractBaseName
$script:selectedPath) + "_extracted")
        return
    }
    $txtTarget.Text = Join-Path $parent ((Get-DefaultArchiveBaseName
$script:selectedPath $script:selectedType) + ".star")
}

function Set-SelectedPath {
    param([string]$Path,[ValidateSet("File","Folder")][string]$Type)
    $script:selectedPath = $Path
    $script:selectedType = $Type
    $lblSelected.Text = "Selected: $script:selectedPath"
    if ($Type -eq "File") {
        if (Is-SmartArchivePath $Path) {
            $btnFile.BackColor = $cBg
            $btnArchive.BackColor = [System.Drawing.Color]::LightBlue
        } else {
            $btnFile.BackColor = [System.Drawing.Color]::LightBlue
            $btnArchive.BackColor = $cBg
        }
        $btnFolder.BackColor = $cBg
    } else {
        $btnFile.BackColor = $cBg
        $btnArchive.BackColor = $cBg
        $btnFolder.BackColor = [System.Drawing.Color]::LightBlue
    }
    Set-DefaultTarget
}

# ================================================================================
# 12. GUI construction
# ================================================================================
$form = New-UiObject "System.Windows.Forms.Form" @{
    Text            = "SmartTAR STAR 1.0 Beta 1 Fix 2 XZ9 XZStable - Clean"
    ClientSize      = (New-Size 505 455)
    StartPosition   = "CenterScreen"
    BackColor       = $cBg
    FormBorderStyle = "FixedSingle"
    MaximizeBox     = $false
    TopMost         = $false
}

$lblInput    = New-EcoLabel "1. Select input file or folder:" 20 20 -Font $fBold
$btnFile     = New-EcoButton "Add FILE" 20 48 150 30
$btnFolder   = New-EcoButton "Add FOLDER" 177 48 150 30
$btnArchive  = New-EcoButton "Add ARCHIVE" 334 48 151 30
$lblSelected = New-EcoLabel "Selected: none" 20 88 465 20 $fItalic
```

```
([System.Drawing.Color]::DimGray)

$lblTarget = New-EcoLabel "2. Destination archive / extraction folder:" 20 125
-Font $fBold
$txtTarget = New-UiObject "System.Windows.Forms.TextBox" @{
    Location  = (New-Point 20 153)
    Size      = (New-Size 395 23)
    Font      = $fNormal
    ReadOnly  = $true
    BackColor = $cBg
}
$btnTarget = New-EcoButton "..." 422 152 63 24

$lblMode = New-EcoLabel "3. Compression mode:" 20 195 -Font $fBold
$cmbMode = New-UiObject "System.Windows.Forms.ComboBox" @{
    Location      = (New-Point 20 223)
    Size          = (New-Size 465 24)
    Font          = $fNormal
    DropDownStyle = [System.Windows.Forms.ComboBoxStyle]::DropDownList
}
[void]$cmbMode.Items.Add("Hybrid - recommended, balanced planner")
[void]$cmbMode.Items.Add("Smart - detailed grouped blocks")
[void]$cmbMode.Items.Add("Solid - one auto-selected block")
[void]$cmbMode.Items.Add("Smart XZ - grouped XZ9 blocks")
$cmbMode.SelectedIndex = 0

$lblInfo      = New-EcoLabel "XZ/XZ9 blocks use fixed timestamps; STORE and ZSTD
blocks keep natural timestamps" 20 252 465 20 $fItalic
([System.Drawing.Color]::DimGray)
$btnCompress  = New-EcoButton "COMPRESS" 20 287 150 42 $fBold
([System.Drawing.Color]::SeaGreen) ([System.Drawing.Color]::White)
$btnExtract   = New-EcoButton "EXTRACT" 177 287 150 42 $fBold
([System.Drawing.Color]::SteelBlue) ([System.Drawing.Color]::White)
$btnVerify    = New-EcoButton "VERIFY" 334 287 151 42 $fBold
([System.Drawing.Color]::DarkSlateGray) ([System.Drawing.Color]::White)
$chkOpenFolder = New-EcoCheck "Open output folder after success" 20 342 300
$true
$lblStatus    = New-EcoLabel "Ready." 20 382 465 20 $fItalic
([System.Drawing.Color]::DimGray)
$progressBar  = New-UiObject "System.Windows.Forms.ProgressBar" @{
    Location              = (New-Point 20 425)
    Size                  = (New-Size 465 8)
    Style                 = [System.Windows.Forms.ProgressBarStyle]::Marquee
    MarqueeAnimationSpeed = 25
    Visible               = $false
}

$form.Controls.AddRange([System.Windows.Forms.Control[]]@(
    $lblInput,$btnFile,$btnFolder,$btnArchive,$lblSelected,
    $lblTarget,$txtTarget,$btnTarget,
    $lblMode,$cmbMode,$lblInfo,
    $btnCompress,$btnExtract,$btnVerify,
    $chkOpenFolder,$lblStatus,$progressBar
```

```
))

# ================================================================================
# 13. GUI events and execution handlers
# ================================================================================
$cmbMode.Add_SelectedIndexChanged({
    $mode = Get-SelectedCompressionMode
    if ($mode -eq "Solid") {
        $lblInfo.Text = "Solid = one block; XZ selections use XZ9 and fixed
timestamps"
    } elseif ($mode -eq "SmartXZ") {
        $lblInfo.Text = "Smart XZ = grouped XZ9 blocks with fixed timestamps;
media/archive STORE"
    } elseif ($mode -eq "Smart") {
        $lblInfo.Text = "Smart = XZ parts use XZ9/fixed timestamps;
binary/exe/diskimage prefer ZSTD19"
    } else {
        $lblInfo.Text = "Hybrid = compressible XZ9/fixed timestamps, disk images
ZSTD19, media/archive STORE"
    }
})

$btnFile.Add_Click({
    $dialog = New-Object System.Windows.Forms.OpenFileDialog
    $dialog.Title = "Select file"
    $dialog.Filter = "All files (*.*)|*.*"
    try { if ($dialog.ShowDialog() -eq [System.Windows.Forms.DialogResult]::OK)
{ Set-SelectedPath $dialog.FileName "File" } }
    finally { $dialog.Dispose() }
})

$btnFolder.Add_Click({
    $dialog = New-Object System.Windows.Forms.FolderBrowserDialog
    $dialog.Description = "Select folder to archive"
    try { if ($dialog.ShowDialog() -eq [System.Windows.Forms.DialogResult]::OK)
{ Set-SelectedPath $dialog.SelectedPath "Folder" } }
    finally { $dialog.Dispose() }
})

$btnArchive.Add_Click({
    $dialog = New-Object System.Windows.Forms.OpenFileDialog
    $dialog.Title = "Select SmartTAR archive"
    $dialog.Filter = "SmartTAR Archive (*.star)|*.star|Legacy SmartTAR Archive
(*.sarc.tar)|*.sarc.tar|All files (*.*)|*.*"
    try { if ($dialog.ShowDialog() -eq [System.Windows.Forms.DialogResult]::OK)
{ Set-SelectedPath $dialog.FileName "File" } }
    finally { $dialog.Dispose() }
})

$btnTarget.Add_Click({
    if ($script:selectedType -eq "File" -and (Is-SmartArchivePath
$script:selectedPath)) {
        $dialog = New-Object System.Windows.Forms.FolderBrowserDialog
```

```
        $dialog.Description = "Select extraction folder"
        try { if ($dialog.ShowDialog() -eq
[System.Windows.Forms.DialogResult]::OK) { $txtTarget.Text =
$dialog.SelectedPath } }
        finally { $dialog.Dispose() }
        return
    }
    $dialog = New-Object System.Windows.Forms.SaveFileDialog
    $dialog.Title = "Select destination archive"
    $dialog.Filter = "SmartTAR Archive (*.star)|*.star|Legacy SmartTAR Archive
(*.sarc.tar)|*.sarc.tar|All files (*.*)|*.*"
    $dialog.DefaultExt = "star"
    $dialog.AddExtension = $true
    $dialog.OverwritePrompt = $true
    try { if ($dialog.ShowDialog() -eq [System.Windows.Forms.DialogResult]::OK)
{ $txtTarget.Text = Ensure-StarExtension $dialog.FileName } }
    finally { $dialog.Dispose() }
})

function Execute-Compress {
    if (-not (Test-Path -LiteralPath $tarPath)) { Msg "tar.exe was not found."
"Missing TAR Engine" ([System.Windows.Forms.MessageBoxIcon]::Error) | Out-Null;
return }
    if (Is-Blank $script:selectedPath) { Msg "Please select a file or folder
first." "Missing Input" ([System.Windows.Forms.MessageBoxIcon]::Warning) |
Out-Null; return }
    if (Is-Blank $txtTarget.Text) { Msg "Please select destination archive
path." "Missing Destination" ([System.Windows.Forms.MessageBoxIcon]::Warning) |
Out-Null; return }
    if (-not (Test-Path -LiteralPath $script:selectedPath)) { Msg "Selected
input path does not exist." "Input Error"
([System.Windows.Forms.MessageBoxIcon]::Error) | Out-Null; return }

    $targetPath = Ensure-StarExtension ($txtTarget.Text.Trim('"'))
    $txtTarget.Text = $targetPath
    $targetDir = [System.IO.Path]::GetDirectoryName($targetPath)
    if (Is-Blank $targetDir) {
        $targetDir = $scriptDir
        $targetPath = Join-Path $targetDir
([System.IO.Path]::GetFileName($targetPath))
        $txtTarget.Text = $targetPath
    }
    if (-not (Test-Path -LiteralPath $targetDir)) { Msg "Destination directory
does not exist:`n$targetDir" "Destination Error"
([System.Windows.Forms.MessageBoxIcon]::Error) | Out-Null; return }

    if (Test-Path -LiteralPath $targetPath) {
        $confirm = Msg "Target archive already exists. Overwrite it?" "Overwrite
Archive" ([System.Windows.Forms.MessageBoxIcon]::Warning)
([System.Windows.Forms.MessageBoxButtons]::YesNo)
        if ($confirm -ne [System.Windows.Forms.DialogResult]::Yes) { return }
    }
```

```
    $mode = Get-SelectedCompressionMode
    Start-UiWork
    $success = $false
    $errorMessage = $null
    try {
        Compress-SmartArchive $tarPath $script:selectedPath $targetPath $mode
        $success = $true
    } catch {
        $errorMessage = Get-ErrorDetails $_
    } finally {
        Stop-UiWork
    }

    if ($success) {
        Set-AppStatus "Archive created successfully. Mode: $mode."
([System.Drawing.Color]::Green)
        $reportPath = Get-ReportPath $targetPath "create_report"
        try {
            $summary = Get-ArchiveSummary $tarPath $targetPath
$script:selectedPath
            [void](Write-ReportFile $reportPath $summary)
            Msg "$summary`r`nReport saved:`r`n$reportPath" "Archive Summary"
([System.Windows.Forms.MessageBoxIcon]::Information) | Out-Null
        } catch {
            $verifyError = Get-ErrorDetails $_
            $fallback = "Archive created successfully, but automatic
summary/verify failed.`r`n`r`nArchive: $targetPath`r`nMode: $mode`r`nSource:
$($script:selectedPath)`r`nArchive size: $(Format-Bytes ([int64](Get-Item
-LiteralPath $targetPath).Length))`r`n`r`nVerify/report error:`r`n$verifyError"
            [void](Write-ReportFile $reportPath $fallback)
            Msg "$fallback`r`nReport saved:`r`n$reportPath" "Archive Created -
Verify Report Failed" ([System.Windows.Forms.MessageBoxIcon]::Warning) |
Out-Null
        }
        if ($chkOpenFolder.Checked) { explorer.exe "/select,`"$targetPath`"" }
        return
    }

    Set-AppStatus "Compression failed." ([System.Drawing.Color]::Red)
    Msg "Compression failed.`n`n$errorMessage" "SmartTAR Error"
([System.Windows.Forms.MessageBoxIcon]::Error) | Out-Null
}

function Execute-Extract {
    if (Is-Blank $script:selectedPath) { Msg "Please select a SmartTAR archive
first." "Missing Archive" ([System.Windows.Forms.MessageBoxIcon]::Warning) |
Out-Null; return }
    if ($script:selectedType -ne "File") { Msg "Extraction input must be a .star
file." "Invalid Input" ([System.Windows.Forms.MessageBoxIcon]::Warning) |
Out-Null; return }
    $destination = $txtTarget.Text.Trim('"')
    if (Is-Blank $destination) { Msg "Please select extraction folder." "Missing
Destination" ([System.Windows.Forms.MessageBoxIcon]::Warning) | Out-Null; return
```

```
}
    if (-not (Test-Path -LiteralPath $destination)) { New-Item -ItemType
Directory -Path $destination -Force | Out-Null }

    Start-UiWork
    $success = $false
    $errorMessage = $null
    try {
        Extract-SmartArchive $tarPath $script:selectedPath $destination
        $success = $true
    } catch {
        $errorMessage = Get-ErrorDetails $_
    } finally {
        Stop-UiWork
    }

    $reportPath = Get-ReportPath $script:selectedPath "extract_report"
    if ($success) {
        Set-AppStatus "Archive extracted successfully."
([System.Drawing.Color]::Green)
        $text = "Archive extracted successfully.`r`nArchive:
$($script:selectedPath)`r`nDestination: $destination"
        [void](Write-ReportFile $reportPath $text)
        Msg "$text`r`nReport saved:`r`n$reportPath" "Extract Archive"
([System.Windows.Forms.MessageBoxIcon]::Information) | Out-Null
        if ($chkOpenFolder.Checked) { explorer.exe "`"$destination`"" }
        return
    }

    [void](Write-ReportFile $reportPath "Extraction
failed.`r`n`r`n$errorMessage")
    Set-AppStatus "Extraction failed." ([System.Drawing.Color]::Red)
    Msg "Extraction failed.`n`n$errorMessage`n`nReport saved:`n$reportPath"
"SmartTAR Error" ([System.Windows.Forms.MessageBoxIcon]::Error) | Out-Null
}

function Execute-Verify {
    if (Is-Blank $script:selectedPath -or $script:selectedType -ne "File" -or
-not (Test-Path -LiteralPath $script:selectedPath)) {
        Msg "Please select an existing .star archive." "Invalid Archive"
([System.Windows.Forms.MessageBoxIcon]::Warning) | Out-Null
        return
    }

    Start-UiWork
    $success = $false
    $errorMessage = $null
    $summary = $null
    try {
        $summary = Verify-SmartArchive $tarPath $script:selectedPath
        $success = $true
    } catch {
        $errorMessage = Get-ErrorDetails $_
```

```
    } finally {
        Stop-UiWork
    }

    $reportPath = Get-ReportPath $script:selectedPath "verify_report"
    if ($success) {
        [void](Write-ReportFile $reportPath $summary)
        Set-AppStatus "Archive verification finished."
([System.Drawing.Color]::Green)
        Msg "$summary`r`nReport saved:`r`n$reportPath" "Verify Archive"
([System.Windows.Forms.MessageBoxIcon]::Information) | Out-Null
        return
    }

    [void](Write-ReportFile $reportPath "Verification
failed.`r`n`r`n$errorMessage")
    Set-AppStatus "Verification failed." ([System.Drawing.Color]::Red)
    Msg "Verification failed.`n`n$errorMessage`n`nReport saved:`n$reportPath"
"SmartTAR Error" ([System.Windows.Forms.MessageBoxIcon]::Error) | Out-Null
}

$btnCompress.Add_Click({ Execute-Compress })
$btnExtract.Add_Click({ Execute-Extract })
$btnVerify.Add_Click({ Execute-Verify })

$form.Add_FormClosing({
    $fNormal.Dispose()
    $fBold.Dispose()
    $fItalic.Dispose()
})

[System.Windows.Forms.Application]::Run($form)
```